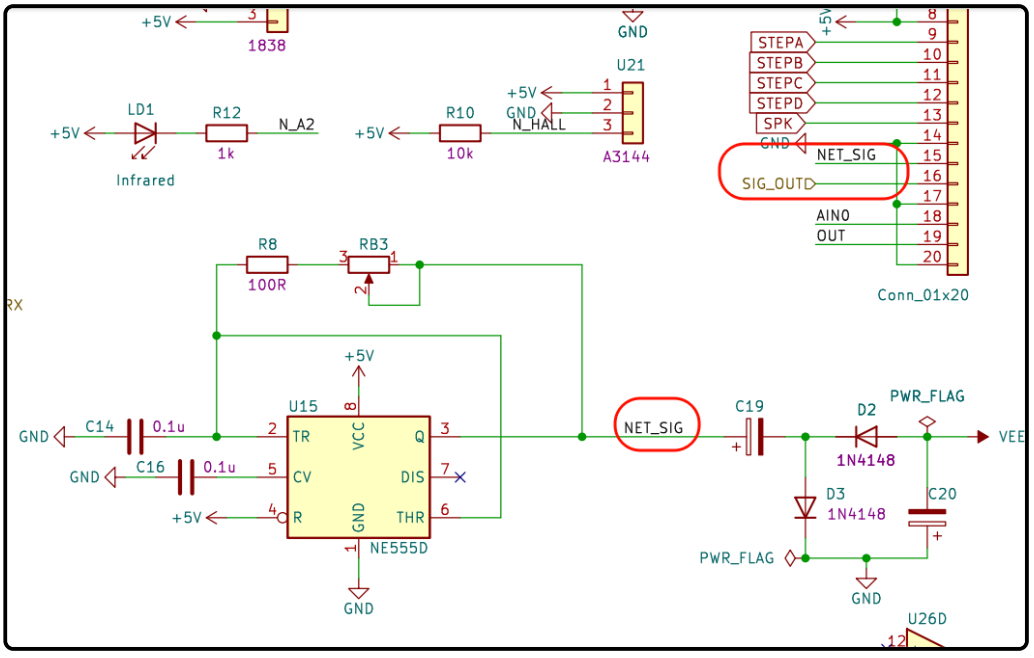
NE555测量底层

原理

对于我们频率测量，我们其实本质上就是对脉冲上升沿进行计数，然后1s内上升沿的个数就是我们的频率

那么我们要用什么进行计数呢，通过查看原理图我们知道，NET_SIG这个就是我们的NE555的输出，然后对于外部引脚我们发现，在Net_SIG旁边的就是SIG_OUT（对应板子上的丝印就是P34）那么我们使用跳线帽连接上这两个东西就行了，就能通过P34引脚的高低电平变化进行测量。



我们查看手册可以知道，定时器0计数对应的就是P34引脚（这里不能使用定时器1！定时器1计数对应的是P35）

并且我们测量的原理是，我们1s去获取一次计数结果，然后将计数结果存放到一个变量中，所以我们在这里要使用16位（因为$2^{16} = 65535$，如果使用8位的话只有$2^8 = 256$和我们最大值2.8w+明显不行），并且对应的这个C/T需要拉高，不然没法作为计数器，所以我们定时器0的这边就可以说是0101，我们不能影响到定时器1的功能，所以我们直接`TMOD |= 0x05`就行，这样就直接将定时器0的相关数据进行修改，但是没有影响到定时器1的相关配置

寄存器TMOD各位的功能描述

TMOD　　地址：89H　　复位值：00H

不可位寻址

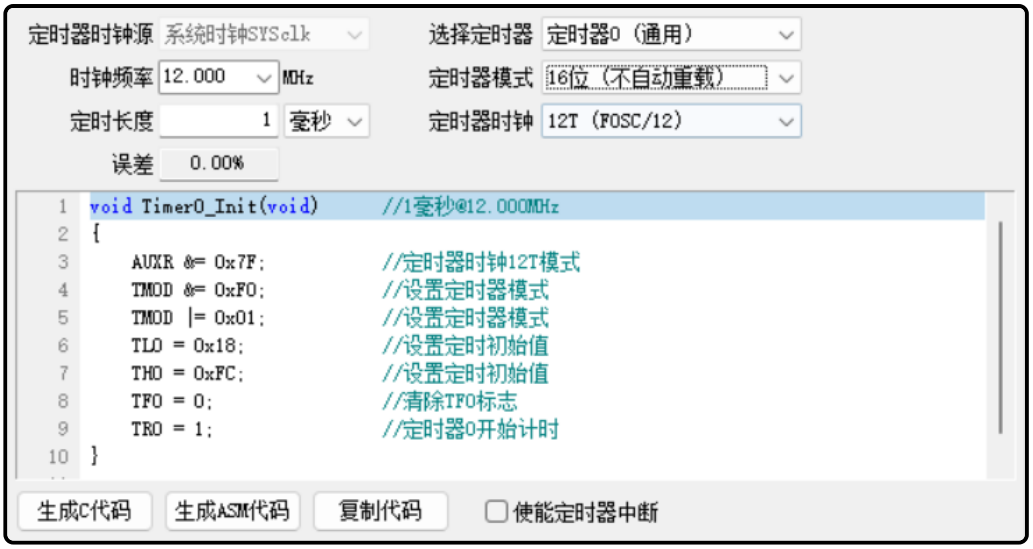
7	6	5	4	3	2	1	0
GATE	C/$\overline{T}$	M1	M0	GATE	C/$\overline{T}$	M1	M0
定时器1				定时器0			

位	符号	功能
TMOD.7/	GATE	TMOD.7控制定时器1，置1时只有在INT1脚为高及TR1控制位置1时才可打开定时器/计数器1。
TMOD.3/	GATE	TMOD.3控制定时器0，置1时只有在INT0脚为高及TR0控制位置1时才可打开定时器/计数器0。
TMOD.6/	C/$\overline{T}$	TMOD.6控制定时器1用作定时器或计数器，清零则用作定时器(对内部系统时钟进行计数)，置1用作计数器(对引脚T1/P3.5外部脉冲进行计数)
TMOD.2/	C/$\overline{T}$	TMOD.2控制定时器0用作定时器或计数器，清零则用作定时器(对内部系统时钟进行计数)，置1用作计数器(对引脚T0/P3.4的外部脉冲进行计数)
TMOD.5/TMOD.4	M1、M0	定时器定时器/计数器1模式选择
	0　0	16位自动重装定时器，当溢出时将RL_TH1和RL_TL1存放的值自动重装入TH1和TL1中。
	0　1	16位不可重装载模式，TL1、TH1全用
	1　0	8位自动重装载定时器，当溢出时将TH1存放的值自动重装入TL1
	1　1	定时器/计数器1此时无效（停止计数）。
TMOD.1/TMOD.0	M1、M0	定时器/计数器0模式选择
	0　0	16位自动重装定时器，当溢出时将RL_TH0和RL_TL0存放的值自动重装入TH0和TL0中。
	0　1	16位不可重装载模式，TL0、TH0全用
	1　0	8位自动重装载定时器，当溢出时将TH0存放的值自动重装入TL0
	1　1	不可屏蔽中断的16位自动重装定时器

对于定时器的初始化，我们就直接使用ISP生成就行了，这里需要注意的是，新版本不支持生成0ms，就随便生成一个然后改一下初始值就行了，但是一定注意，这些参数必须要保持一致

底层书写

定时器0，12MHz，16位（不自动重载），定时器时钟12T（千万千万别开中断）



```c
void Timer0_Init(void)		//1毫秒@12.000MHz
{
	AUXR &= 0x7F;			//定时器时钟12T模式
	TMOD &= 0xF0;			//设置定时器模式
	TMOD |= 0x05;			//定时器计数，16位不自动重载
	TL0 = 0x00;				//设置定时初始值
	TH0 = 0x00;				//设置定时初始值
	TF0 = 0;				//清除TF0标志
	TR0 = 1;				//定时器0开始计时
}

```

我们在上面已经实现了计数，那么现在就需要将计数的结果取出来，我们首先需要再开一个定时器，然后再中断里面定时1s取数据就行了，注意，1s定时变量需要使用uint，不要写成uchar了，我们在这里使用定时器1开一个1ms的定时器来实现操作

并且注意一下，我们取的时候是TH0和TL0都要拿出来，TH0是高八位，所以我们需要左移8位进行操作

当我们数据取完后，我们就可以将TL0和TH0归0，以便下一次获取

```c
unsigned int time_all_1s;
void Timer1_Init(void)		//1毫秒@12.000MHz
{
	AUXR &= 0xBF;			//定时器时钟12T模式
	TMOD &= 0x0F;			//设置定时器模式
	TL1 = 0x18;				//设置定时初始值
	TH1 = 0xFC;				//设置定时初始值
	TF1 = 0;				//清除TF1标志
	TR1 = 1;				//定时器1开始计时
	ET1 = 1;				//使能定时器1中断
	EA = 1;
}

void Timer1_Isr(void) interrupt 3
{
	if(++time_all_1s==1000)
	{
		time_all_1s=0;
		Freq = TH0 << 8 | TL0;
		TH0 = TL0 = 0;
	}
}
```

NE555易错解析

初始化顺序问题

当我们测量的时候，必须在main函数里面初始化遵循先初始化定时器0，再初始化定时器1的顺序，否则会导致第一次测量的时候频率不对。

```
void main()
{
    Timer0_Init();
    Timer1_Init();
    while(1)
    {

    }
}
```

第一次测量结果误差

由于我们使用测量的时候，第一次测量的数据如果再4T上测的话，会导致最初的那个值不对，我们这里的操作就是直接让数码管1s后显示，我们只需要定义一个bit就行了，然后1s获取完成值后，将这个bit置为1，当bit为1的时候，我们才开始进行数码管显示

```
bit show_flag_first=0;
void Seg_Proc()
{
    if(show_flag_first)
    {
        //进行频率显示
    }
}
void Timer1_Isr(void) interrupt 3
{
    if(++time_all_1s==1000)
    {
         show_flag_first=1;
        time_all_1s=0;
        Freq = TH0 << 8 | TL0;
        TH0 = TL0 = 0;
    }
}
```

接线问题

这里一定要使用跳线帽接上P34和SIGNAL这个线，否则会测量不到东西

按键问题

由于按键最后一列会用到P34，所以我们这里必须把他给屏蔽了，不然会导致P34的上升下降测量误差，会影响评分

```
unsigned char Key_Read()
{
    unsigned char temp = 0;
    P44=0;P42=1;P35=1;//P34=1;
    if(P33==0) temp=4;
    if(P32==0) temp=5;
    if(P31==0) temp=6;
    if(P30==0) temp=7;

```

```
    P44=1;P42=0;P35=1;//P34=1;
    if(P33==0) temp=8;
    if(P32==0) temp=9;
    if(P31==0) temp=10;
    if(P30==0) temp=11;

    P44=1;P42=1;P35=0;//P34=1;
    if(P33==0) temp=12;
    if(P32==0) temp=13;
    if(P31==0) temp=14;
    if(P30==0) temp=15;
    /*
    P44=1;P42=1;P35=1;P34=0;
    if(P33==0) temp=16;
    if(P32==0) temp=17;
    if(P31==0) temp=18;
    if(P30==0) temp=19;
    */
    return temp;
}
```